

Virtual Thread (VT) Transport

Code Review Documentation

Line-by-line comparison against the existing NIO PassThrough transport

WSO2 Synapse | Branch `VTPassTV2`
modules/transport/core/nhttp/.../passthru/vt/

Prepared for code review

Date: 22 April 2026

Table of Contents

1. Executive Summary & Goals
2. High-Level Architecture Shift (NIO → VT)
3. Classes Under `vt/` — Summary Matrix
4. `VTPassThroughHttpListener` vs `PassThroughHttpListener`
5. `VTHttpSender` vs `PassThroughHttpSender`
6. `VTBlockingServerWorker` vs `ServerWorker`
7. `VTInputStreamPipe` vs `Pipe`
8. `VirtualThreadWorkerPool` (new adapter)
9. `VTConstants` vs `PassThroughConstants`
10. Reused Existing Utilities (Cross-Cutting)
11. Deployment Notes & Risks

1. Executive Summary

The Virtual Thread (VT) transport is a fully blocking, Project-Loom-based replacement for the NIO PassThrough transport's request/response plumbing. The goal is to keep a *blocking, easy-to-reason* programming model per request while retaining the scalability of asynchronous I/O — by running each HTTP request on a lightweight Java 21 virtual thread instead of a shared platform-thread pool driven by an NIO reactor.

Why it exists. The existing NIO transport uses `DefaultListeningIOReactor`, asynchronous `ContentDecoder` / `ContentEncoder` callbacks, and a `Pipe` backed by a `ControlledByteBuffer`. This model is fast but complex: back-pressure, keep-alive, and chunked decoding all live in callbacks. VT lets us express the same flow as straight-line code (`socket.read()` → `engine.receive()` → `response.write()`) on a VT that parks instead of pinning a platform thread.

Design constraints that shaped the implementation

- **Reuse Synapse/Axis2 contracts.** Listener still implements `TransportListener`; sender still implements `TransportSender`; worker still implements `OutTransportInfo`. Nothing above the transport layer changes.
- **Reuse `SourceConfiguration` and `TargetConfiguration`.** These are full Axis2 lifecycle objects — recreating them would mean rebuilding JMX, metrics, WorkerPool wiring, and EPR-prefix logic.
- **Delegate HTTP protocol handling to HttpCore 5 blocking API.** Parsing request lines, chunked decoding, keep-alive, and response serialization are too tricky to reimplement. `HttpService` does this and exposes `ClassicHttpRequest` / `ClassicHttpResponse`.
- **Replace HttpCore 4 NIO sender with HttpClient 4.5.x blocking client.** The outbound path needs a blocking client that plays nicely with a VT — `CloseableHttpClient` with `PoolingHttpClientConnectionManager` satisfies this and exposes `InputStreamEntity` for zero-copy body streaming.
- **Zero-copy streaming path.** When no content-aware mediator runs before `<call/>`, the raw client socket `InputStream` is piped directly to the backend via `InputStreamEntity`, avoiding AXIOM build. This mirrors NIO passthrough behaviour.

2. High-Level Architecture Shift

2.1 Request path (inbound)

Stage	NIO PassThrough	VT Transport
Accept	<code>DefaultListeningIOReactor</code> + <code>ServerConnFactory</code>	Blocking <code>ServerSocket</code> on one platform thread
Dispatch	<code>ServerIODispatch</code> → <code>SourceHandler</code> callbacks	<code>vtExecutor.submit(...)</code> → one Virtual Thread per connection
HTTP parsing	Custom: <code>SourceRequest</code> , <code>SourceHandler</code> , <code>ProtocolState</code>	HttpCore 5 <code>HttpService.handleRequest()</code>
Worker invocation	<code>ServerWorker</code> posted to <code>WorkerPool</code>	<code>VTBlockingServerWorker.process()</code> called synchronously from same VT
Body transport	<code>Pipe</code> (NIO <code>ControlledByteBuffer</code> , producer/consumer)	<code>VTInputStreamPipe</code> (wraps socket <code>InputStream</code>)
Engine dispatch	<code>AxisEngine.receive()</code>	<code>AxisEngine.receive()</code> — unchanged

2.2 Response path (outbound back to client)

Stage	NIO PassThrough	VT Transport
Entry	<code>PassThroughHttpSender.invoke()</code> → <code>submitResponse()</code>	<code>VTHttpSender.invoke()</code> → <code>submitResponse()</code>
Response object	<code>SourceResponse</code> / <code>SourceResponseFactory</code>	<code>ClassicHttpResponse</code> populated directly by worker
Wire write	NIO <code>ContentEncoder</code> callbacks	HttpCore 5 writes response when handler returns (on same VT)

2.3 Backend call path (CallMediator blocking)

Stage	NIO PassThrough	VT Transport
Sender	Built-in <code>http</code> / <code>https</code> transport based on <code>HTTPSender</code> (Commons HttpClient 3.x)	<code>VTHttpSender</code> with HttpClient 4.5.x pooled client
Zero-copy body	Via <code>Pipe</code> only in non-blocking path	Via <code>InputStreamEntity(vtPipe.getInputStream(), contentType)</code>
Pool	<code>MultiThreadedHttpConnectionManager</code> (deprecated)	<code>PoolingHttpClientConnectionManager</code>
Timeouts	<code>HttpConnectionManagerParams</code>	<code>RequestConfig</code>

3. Classes Under vt/ — Summary Matrix

VT Class	Replaces / Counterpart	Lines	Nature of Change
<code>VTPassThroughHttpListener</code>	<code>PassThroughHttpListener</code> (716 LOC)	472	Rewrite — swap NIO reactor for <code>ServerSocket</code> + VT executor + <code>HttpCore 5</code>
<code>VTHttpSender</code>	<code>PassThroughHttpSender</code> (786 LOC)	826	Unified sender — <code>HttpClient 4.5.x</code> , zero-copy streaming, passthrough response merged
<code>VTBlockingServerWorker</code>	<code>ServerWorker</code> (848 LOC)	785	Rewrite — driven by <code>HttpCore 5</code> request handler, <code>OutTransportInfo</code>
<code>VTInputStreamPipe</code>	<code>Pipe</code> (756 LOC)	50	Replacement — VT blocks, so we only need a wrapper around <code>InputStream</code>
<code>VirtualThreadWorkerPool</code>	(new adapter)	77	Axis2 <code>WorkerPool</code> impl backed by <code>Executors.newThreadPerTaskExecutor</code>
<code>VTConstants</code>	<code>PassThroughConstants</code> (314 LOC, still reused)	120	Small additive — VT-only property keys, socket params, HTTP context attrs

4. `VTPassThroughHttpListener` VS `PassThroughHttpListener`

File `modules/transport/core/nhttp/src/main/java/org/apache/synapse/transport/passthru/vt/VTPassThroughHttpListener.java`
Implements `org.apache.axis2.transport.TransportListener` — unchanged contract
Axis2 role Accepts inbound HTTP connections and dispatches each request into the Axis2/Synapse mediation engine

4.1 Import changes

Import	<code>PassThrough (NIO)</code>	VT	Why changed
IO reactor	<code>DefaultListeningIOReactor</code> , <code>IOReactorException</code> , <code>IOReactorExceptionHandler</code>	— removed —	No reactor in VT; accept loop is a plain <code>while (running.get())</code> thread
Sockets	(none — reactor owns socket channels)	<code>java.net.ServerSocket</code> , <code>Socket</code> , <code>ServerSocketFactory</code> , <code>InetSocketAddress</code>	VT uses classic blocking BSD sockets; each accepted <code>Socket</code>
HTTP protocol	<code>HttpConnectionParams</code> from <code>org.apache.http.params</code>	<code>org.apache.hc.core5.http.impl.io.HttpService</code> , <code>DefaultBHttpServerConnection</code> , <code>ClassicHttpRequest/Response</code> , <code>HttpProcessors</code> , <code>BasicHttpServerRequestHandler</code> , <code>Http1Config</code>	<code>HttpCore 5</code> blocking API replaces the <code>HttpCore 4</code> NIO pipeline. <code>HttpService.handleRequest()</code> handles parsing, chunking, and response serialization — none of which we need to write.
Executor	<code>NativeThreadFactory</code> (fixed-size platform pool via <code>Axis2 WorkerPool</code>)	<code>java.util.concurrent.ExecutorService</code> , <code>Executors</code> , <code>AtomicBoolean</code>	Pool is now <code>Executors.newThreadPerTaskExecutor(Thread.ofVirtual())</code> . Each accepted connection gets a fresh VT.
Conn factory / interceptors	<code>ServerConnFactory</code> , <code>ServerConnFactoryBuilder</code> , <code>StreamInterceptorsLoader</code>	— removed —	<code>HttpCore 5</code> does connection construction internally via <code>DefaultBHttpServerConnection</code> . Stream interceptors (if any) can be added later as <code>HttpCore 5</code> interceptors — not in scope for VT.
JMX	<code>MBeanRegistrar</code> , <code>TransportView</code> , <code>ManagementSupport</code> , <code>TransportMBeanSupport</code>	— removed —	Kept out of the first cut. Metrics collector still instantiated so <code>TransportView</code> remains well-formed, but MBean registration is deferred until <code>HttpCore 5</code> is ready.
Active-conn monitor	<code>ActiveConnectionMonitor</code> , <code>ScheduledExecutorService</code>	— removed —	NIO needed it to pause the reactor when <code>maxActiveConnections</code> reached. Under VT each connection has its own lightweight thread, so <code>maxActiveConnections</code> is irrelevant; rate limiting can be expressed as a <code>Semaphore</code> limit.
Service tracker	<code>AxisServiceTracker</code> , <code>AxisServiceFilter</code> , <code>AxisServiceTrackerListener</code>	REUSED same classes	Lifecycle-driven EPR map updates are transport-agnostic, so reuse.

4.2 New / removed fields

ADDED VT-specific fields

```
private ServerSocket serverSocket;           // blocking accept
private ExecutorService vtExecutor;         // per-task VT executor
private final AtomicBoolean running = new AtomicBoolean(false); // stop flag
private Thread acceptThread;                // single platform thread running acceptLoop()
private HttpService httpService;            // HttpCore 5 blocking service
private int soTimeout = VTConstants.DEFAULT_SO_TIMEOUT;
private int backlog = VTConstants.DEFAULT_BACKLOG;
private boolean tcpNoDelay = VTConstants.DEFAULT_TCP_NODELAY;
```

REMOVED NIO fields

```
private DefaultListeningIOReactor ioReactor;
private PassThroughListeningIOReactorManager passThroughListeningIOReactorManager;
private SourceHandler handler;
private ServerConnFactory connFactory;
private List<StreamInterceptor> interceptors;
private final ScheduledExecutorService activeConnectionMonitorScheduler;
```

REUSED identical fields

```
private SourceConfiguration sourceConfiguration; // same type, same role
private Scheme scheme;
private String namePrefix;
private int operatingPort;
private ConfigurationContext configurationContext;
private TransportInDescription transportInDescription;
private Map<String, String> serviceNameToEPRMap = new HashMap<>();
private Map<String, String> eprToServiceNameMap = new HashMap<>();
private AxisServiceTracker serviceTracker;
private volatile int state = BaseConstants.STOPPED;
```

4.3 Function-level comparison

`init(ConfigurationContext, TransportInDescription)` — significantly simplified

Step	PassThrough	VT
Port resolution (offset)	KEPT Same 4-line pattern: read <code>port</code> param, add <code>portOffset</code> sysprop, set back into param	KEPT IDENTICAL — parameter key moved into <code>VTConstants.PARAM_PORT</code>
Sysprop advertisement	<code>{name}.nio.port</code>	<code>{name}.VT.port</code> — so existing NIO-assuming consumers don't collide
WorkerPool	Reuses <code>PASS_THROUGH_TRANSPORT_WORKER_POOL</code> if present, else creates <code>NativeWorkerPool</code>	Reuses the same context key if present; else creates <code>new VirtualThreadWorkerPool(vtExecutor)</code>
Metrics	<code>new PassThroughTransportMetricsCollector(true, scheme.getName())</code>	Same — REUSED
SourceConfiguration build	<code>new SourceConfiguration(cfgCtx, transportIn, scheme, workerPool, metrics); build();</code>	Identical — REUSED as-is. Keeps JMX, EPR-prefix, transport-param parsing working.
Connection factory	<code>initConnFactoryBuilder(...).build(...)</code>	REMOVED — HttpCore 5 builds its own <code>DefaultBHttpClientConnection</code> per socket
IOReactor init	<code>passThroughListeningIOReactorManager.initIOReactor(...)</code> wrapped in <code>try/catch IOException</code>	REMOVED
HttpService build	—	ADDED — <code>HttpService(HttpProcessors.server(), new BasicHttpRequestHandler(mapper))</code> . The mapper is a lambda returning a <code>HttpRequestHandler</code> that constructs a <code>VTBlockingServerWorker</code> for every request.
EPR map bootstrap	Same <code>HashMap</code> init + <code>EPR_TO_SERVICE_NAME_MAP</code> context property	Identical logic — REUSED
MBean registration	<code>MBeanRegistrar.getInstance().registerMBean(view, "Transport", ...)</code>	REMOVED — deferred (see §4.1)
Service tracker	Same construction with filter + listener	IDENTICAL

`start()` — accept loop replaces reactor start

`PassThrough` calls `ioReactor.setExceptionHandler(...)`, then `startPTTEndpoint(address, ioReactor, namePrefix)` for each bind address — this schedules the reactor on its own thread. VT instead:

```
serverSocket = createServerSocket(bindAddress);
running.set(true);
state = BaseConstants.STARTED;
acceptThread = new Thread(this::acceptLoop, VTConstants.VT_THREAD_PREFIX + namePrefix + "-accept");
acceptThread.setDaemon(true);
acceptThread.start();
```

The accept thread is a *single* platform thread; handing off the accepted socket to a VT means it never blocks doing real work.

NEW acceptLoop()

```
while (running.get()) {
    Socket clientSocket = serverSocket.accept();
    clientSocket.setSoTimeout(soTimeout);
    clientSocket.setTcpNoDelay(tcpNoDelay);
    vtExecutor.submit(() -> handleConnection(clientSocket));
}
```

Why: this is the simplest possible accept pattern — the entire NIO reactor is replaced by one `accept()` + one VT dispatch. The `IOException` check after accept distinguishes normal shutdown (caused by `serverSocket.close()` in `stop()`) from actual errors.

NEW handleConnection(Socket)

```

Http1Config h1Config = Http1Config.custom().setBufferSize(VTConstants.STREAM_BUFFER_SIZE).build();
conn = new DefaultBHttpServerConnection(scheme.getName(), h1Config);
conn.bind(clientSocket);
BasicHttpContext context = new BasicHttpContext();
context.setAttribute(VTConstants.CTX_REMOTE_ADDRESS, clientSocket.getRemoteSocketAddress());
context.setAttribute(VTConstants.CTX_LOCAL_ADDRESS, clientSocket.getLocalSocketAddress());
context.setAttribute(VTConstants.CTX_LOCAL_PORT, clientSocket.getLocalPort());
while (conn.isOpen()) {
    httpService.handleRequest(conn, context);
}

```

Why the loop: HTTP keep-alive. `handleRequest()` processes exactly one request/response cycle and returns. HttpCore 5 keeps the connection open based on `Connection: / HTTP/1.1` semantics; we simply keep calling until the socket is dead. The remote / local address is stashed in `HttpContext` so the worker can retrieve it without holding a socket reference.

CHANGED `stop()`

PassThrough shuts down the reactor and endpoints via `PassThroughListeningIOReactorManager`. VT instead:

- `Flip running.set(false)`
- `serverSocket.close()` — unblocks the accept loop
- `vtExecutor.shutdown()` — existing VTs drain naturally
- `acceptThread.join(5000)` — bounded wait
- `serviceTracker.stop()` — same as before

IDENTICAL `getEPRForService()`, `getEPRsForService()`, `getSessionContext()`

Copy-for-copy from the PassThrough listener — these are transport-agnostic.

IDENTICAL `addToServiceURIMap()`, `removeServiceFromURIMap()`

Same logic as PassThrough's private helpers (`removeServiceFfromURIMap` <typo in original> corrected to `removeServiceFromURIMap`).

4.4 Functions removed from PassThrough counterpart (and why)

Function	Why not needed under VT
<code>startEndpoints()</code> , <code>startSpecificEndpoints(Set<InetSocketAddress>)</code>	They iterate bind addresses and call <code>passThroughListeningIOReactorManager.startPTTEndpoint(...)</code> — reactor-only concept.
<code>addMaxActiveConnectionCountController(int)</code>	Pauses the reactor on threshold; VT has no reactor to pause.
<code>isVersionedBasedService</code> , <code>isServiceWithCustomURI</code> , plus the long version-aware branch inside <code>getEPRsForService()</code>	Defer to existing PassThrough listener for versioned services; the VT transport targets the plain-EPR case for the prototype.
<code>reloadSSLConfig()</code> , <code>reloadDynamicEndpoints()</code>	SSL hot-reload and dynamic inbound endpoints require the reactor manager infrastructure; not in first-cut scope.
Correlation-log plumbing (<code>CorrelationConstants</code>)	Not wired into the VT path yet — can be reintroduced inside <code>handleConnection</code> when required.

5. VTHttpSender VS PassThroughHttpSender

File modules/transport/core/nhttp/src/main/java/org/apache/synapse/transport/passthru/vt/VTHttpSender.java
Extends / impl `AbstractHandler`, `TransportSender` — same as PassThrough
Registered in `axis2.xml` and `axis2_blocking_client.xml` — one class serves both outbound paths

Important: `VTHttpSender` is the *merged* class that replaces the original `VTPassThroughHttpSender` (deleted in earlier commit). It handles three separate cases in a single file:

1. Backend calls (`CallMediator` blocking path) — fallback via `HTTPSender.send()`
2. Backend calls — **streaming fast path** via HttpClient 4.5.x `InputStreamEntity`
3. Passthrough response back to client — via `VTBlockingServerWorker.submitResponse()`

5.1 Import changes

Area	PassThroughHttpSender	VTHttpSender	Why
NIO reactor / target plumbing	<code>DefaultConnectingIOReactor</code> , <code>ClientConnFactory</code> , <code>ClientIODispatch</code> , <code>TargetHandler</code> , <code>TargetConnections</code> , <code>DeliveryAgent</code> , <code>ConnectCallback</code> , <code>ProxyConfig</code> + <code>ProxyConfigBuilder</code>	— all removed —	VT is blocking; there's no persistent I/O loop to dispatch connections. HttpClient 4.5.x owns its own blocking socket pool.
HttpClient	(none — used NIO <code>HttpAsyncClient</code> - style path)	<code>org.apache.http.impl.client.CloseableHttpClient</code> , <code>HttpClient</code> , <code>HttpClientBuilder</code> , <code>PoolingHttpClientConnectionManager</code> , <code>RequestConfig</code> , <code>RequestBuilder</code> , <code>InputStreamEntity</code> , <code>ContentType</code> , <code>CloseableHttpResponse</code> , <code>Header</code> , <code>HttpRequestBase</code>	HttpClient 4.5.x is the synchronous, pooled, blocking client we use for the actual backend HTTP call.
Axis2 HTTP fallback	n/a	<code>HTTPSender</code> , <code>AbstractHTTPSender</code> , <code>ServletBasedOutTransportInfo</code> , <code>AxisHttpResponseImpl</code>	The OM-tree fallback path in <code>writeMessageWithCommons()</code> reuses Axis2's <code>HTTPSender</code> . The Servlet / Axis response objects are for the output-stream path.
Caching, correlation, stream interceptors, SSL hot-reload	<code>CachingConstants</code> , <code>DigestGenerator</code> , <code>CorrelationConstants</code> , <code>StreamInterceptorsLoader</code> , <code>SynapseDebugInfoHolder</code> , <code>RelayUtils</code> , <code>SourceResponseFactory</code>	— removed —	All tangential to the core send path. Response is no longer built via <code>SourceResponseFactory</code> — the <code>VTBlockingServerWorker</code> writes the response directly on the <code>ClassicHttpResponse</code> .
Utils kept	<code>AddressingHelper</code> , <code>PassThroughTransportUtils</code> , <code>TransportUtils</code> , <code>HTTPConstants</code> , <code>HTTPTransportUtils</code> , <code>JavaUtils</code> , <code>MessageProcessorSelector</code>	ALL REUSED	Content-type detection, MTOM/SWA flags, SOAPAction resolution, and the "remove unwanted headers" policy belong to Axis2; no need to reimplement.

5.2 New / removed fields

ADDED

```
private PoolingHttpClientConnectionManager connectionManager; // pool for HttpClient 4.5.x
private Scheme scheme;
private TargetConfiguration targetConfiguration; // reuses PassThrough's TargetConfiguration
private volatile int state = BaseConstants.STOPPED;
```

REMOVED

```
private DefaultConnectingIOReactor ioReactor;
private TargetHandler handler;
private ClientIODispatch ioEventDispatch;
private ClientConnFactory connFactory;
private DeliveryAgent deliveryAgent;
private ProxyConfig proxyConfig;
private TargetConnections targetConnections;
private DigestGenerator digestGenerator;
private List<StreamInterceptor> interceptors;
```

5.3 Function-level comparison

```
init(ConfigurationContext, TransportOutDescription)
```

Step	PassThrough	VT	Why changed
Log banner	"Initializing Pass-through HTTP/S Sender..."	Equivalent VT banner	Cosmetic
HTTP version / chunked default	— (handled elsewhere)	Explicit parsing of <code>PROTOCOL_VERSION</code> and <code>Transfer-Encoding</code> params with fail-closed default	VTHttpSender builds its own request so it needs to know these up-front; PassThrough relied on per-request NHTTP logic
Timeouts	Stored in <code>TargetConfiguration</code> , applied by NHTTP reactor	Stored in fields <code>soTimeout</code> , <code>connectionTimeout</code> , then pushed into <code>RequestConfig</code>	HttpClient 4.5.x expects timeouts on <code>RequestConfig</code> , not on connection-manager params
Pool build	Reactor + <code>TargetConnections</code>	<code>PoolingHttpClientConnectionManager</code> with <code>defaultMaxPerRoute</code> and <code>maxTotal</code> sourced from the same transport params (<code>DEFAULT_MAX_CONNECTIONS_PER_HOST</code> , <code>MAX_TOTAL_CONNECTIONS</code>)	HttpClient 4.5 renamed <i>per host</i> → <i>per route</i>
Client cache	Not applicable (reactor-based)	When <code>CACHE_HTTP_CLIENT=true</code> , client is cached under <code>HTTPConstants.CACHED_HTTP_CLIENT</code> on the <code>ConfigurationContext</code>	The streaming path and Axis2 <code>HTTPSender</code> both pull from this property — a single pooled client
TargetConfiguration	Built, stored in <code>targetConfiguration</code>	IDENTICAL BUILD — still needed because <code>PassThroughTransportUtils.removeUnwantedHeaders()</code> reads it	
Sender-manager / JMX / interceptors	<code>PassThroughSenderManager.register(...)</code> , <code>MBeanRegistrar</code> , <code>StreamInterceptorsLoader</code>	REMOVED	First-cut scope
Reactor thread start	Spawns platform thread running <code>ioReactor.execute(...)</code>	REMOVED — HttpClient has no event loop	

`invoke(MessageContext)`

Structurally similar to PassThrough — same overall shape of "clean headers → setup format → resolve EPR → dispatch". Differences:

Concern	PassThrough	VT
Header cleanup	<code>PassThroughTransportUtils.removeUnwantedHeaders(msgContext, targetConfiguration)</code>	IDENTICAL — guarded by null-check because <code>init()</code> might be called in client-only mode with only <code>CACHE_HTTP_CLIENT</code> set
EPR resolution	<code>PassThroughTransportUtils.getDestinationEPR()</code>	Local logic: prefer <code>TRANSPORT_URL</code> else <code>msgContext.getTo()</code> . More explicit; avoids running NHTTP-specific fallbacks.
Backend dispatch	<code>sendRequestContent(msgContext, epr)</code> — schedules writes on the IOReactor	<code>writeMessageWithCommons(msgContext, epr, format)</code> — blocking call. Checks <code>PASS_THROUGH_PIPE</code> / <code>MESSAGE_BUILDER_INVOKED</code> to pick streaming vs OM-tree path
Passthrough response	Instance-of check <code>OUT_TRANSPORT_INFO instanceof ServerWorker</code> → <code>submitResponse(msgContext)</code>	SAME PATTERN but checks <code>instanceof VTBlockingServerWorker</code> → <code>submitResponse(msgContext)</code>
Servlet response fallback	Via <code>sendUsingOutputStream(msgContext)</code>	<code>sendUsingOutputStream(msgContext, format)</code> — same idea, but also handles <code>AxisHttpResponseImpl</code> via a new <code>addCustomHeadersToAxisResponse()</code> helper
Response-written flag	<code>msgContext.getOperationContext().setProperty(RESPONSE_WRITTEN, VALUE_TRUE)</code>	IDENTICAL

NEW `sendStreamedRequest(MessageContext, EndpointReference, VTInputStreamPipe)`

The key optimisation of the VT transport: take the raw client-socket `InputStream` (wrapped in `VTInputStreamPipe`) and shove it into an `InputStreamEntity` on the HttpClient 4 request. No AXIOM tree is built; bytes flow client → backend without ever hitting the JVM heap as objects.


```

CloseableHttpClient httpClient = (CloseableHttpClient) msgContext.getConfigurationContext()
    .getProperty(HTTPConstants.CACHED_HTTP_CLIENT);
if (httpClient == null) throw new AxisFault("VT cached HttpClient not available ...");

RequestBuilder reqBuilder = RequestBuilder.create(httpMethod).setUri(url);

// Copy transport headers, skipping hop-by-hop / HttpClient-managed
for (Map.Entry<String, Object> e : transportHeaders.entrySet())
    if (!isRestrictedHeader(e.getKey()))
        reqBuilder.addHeader(e.getKey(), e.getValue().toString());

if (hasBody) {
    ContentType ct = parseOrDefault(msgContext.getProperty(CONTENT_TYPE));
    reqBuilder.setEntity(new InputStreamEntity(vtPipe.getInputStream(), ct));
}

response = httpClient.execute(reqBuilder.build());
populateResponseOnMessageContext(msgContext, response);

```

Why not use `HTTPSender` always? `HTTPSender` calls `MessageFormatter.writeTo()`, which for passthrough bodies serialises the AXIOM tree. That's exactly what we want to avoid when a pipe is available.

NEW `populateResponseOnMessageContext(MessageContext, CloseableHttpResponse)`

Translates `HttpClient` 4 response → `Axis2 MessageContext` in the same shape the regular `AbstractHTTPSender` does, so downstream `OperationClient.execute()` can build the envelope via its standard builder pipeline:

```

msgCtx.setProperty(HTTPConstants.MC_HTTP_STATUS_CODE, statusCode);
msgCtx.setProperty("transport.http.statusCode", String.valueOf(statusCode));
msgCtx.setProperty(MessageContext.TRANSPORT_HEADERS, respHeaders); // from response.getAllHeaders()
msgCtx.setProperty(Constants.Configuration.CONTENT_TYPE, ctHeader.getValue());
msgCtx.setProperty(MessageContext.TRANSPORT_IN, response.getEntity().getContent());
msgCtx.setProperty("VT_HTTP_RESPONSE", response); // for cleanup()

```

NEW `isRestrictedHeader(String)`

Drops `content-length`, `host`, `connection`, `transfer-encoding` before they are copied to the outbound request. `HttpClient` computes / owns all four — passing them through creates duplicates or mismatches that the backend rejects.

NEW `submitResponse(MessageContext)`

Tiny dispatcher — pulls the `VTBlockingServerWorker` out of `Constants.OUT_TRANSPORT_INFO` and calls `worker.submitResponse(msgContext)`. The worker owns the `ClassicHttpResponse`.

`cleanup(MessageContext)`

`PassThrough`'s implementation is empty. VT reads `HTTPConstants.HTTP_METHOD` and, if it's an `HttpRequestBase`, aborts it to release the pooled connection.

5.4 Removed / simplified functions

PassThrough function	Why gone
<code>sendRequestContent(MessageContext, EndpointReference)</code>	Reactor-driven; replaced by <code>writeMessageWithCommons()</code> + <code>sendStreamedRequest()</code>
<code>sendAsync(...)</code> and NIO callback hooks	Not needed — VT is synchronous
<code>handleException(String, Exception)</code> with internal reactor log	Replaced by direct <code>throw AxisFault.makeFault(e)</code>
<code>reloadDynamicSSLConfig()</code> , <code>reloadProxyConfig()</code>	SSL / proxy hot-reload deferred
<code>init(...)</code> branches that wire <code>PassThroughSenderManager</code> , <code>StreamInterceptors</code> , <code>MBeanRegistrar</code>	Out of scope for first cut; can be added additively

6. `VTBlockingServerWorker` VS `ServerWorker`

File	modules/transport/core/nhttp/src/main/java/org/apache/synapse/transport/passthru/vt/VTBlockingServerWorker.java
Implements	<code>OutTransportInfo</code> — <i>not Runnable</i>
Driven by	<code>HttpCore 5 HttpRequestHandler</code> lambda inside the listener

6.1 Import changes

Area	ServerWorker	VTBlockingServerWorker	Why
HTTP object model	<code>SourceRequest</code> , <code>SourceResponse</code> , <code>SourceResponseFactory</code>	<code>ClassicHttpRequest</code> , <code>ClassicHttpResponse</code> , <code>HttpEntity</code> , <code>ByteArrayEntity</code> , <code>StringEntity</code> , <code>HttpContext</code> , <code>HttpStatus</code>	HttpCore 5 gives the worker the parsed request and a writable response directly; no need for <code>PassThrough</code> 's intermediate objects
Transport plumbing	<code>NHttpServerConnection</code> , <code>SSLIOSession</code> , <code>HttpInetConnection</code> , <code>BasicHttpEntity</code>	— removed —	HttpCore 5 hides the connection. Addresses live in <code>HttpContext</code> attributes set by the listener
Body IO	<code>Pipe</code> reads via NIO producer/consumer	<code>InputStream</code> read synchronously from <code>httpRequest.getEntity().getContent()</code>	VT can block on read; no decoder callbacks
REST / SOAP	<code>RESTUtil</code> , <code>HTTPTransportUtils</code> , <code>BuilderUtil</code> , <code>RequestURIBasedDispatcher</code> , <code>SOAP11Factory</code>	ALL REUSED	The REST content-type handling, SOAP envelope bootstrap and dispatcher are transport-agnostic
Logging / tenancy / correlation	<code>CorrelationConstants</code> , <code>TenantInfoInitiator</code> , <code>CustomLogSetter</code> , <code>SynapseDebugInfoHolder</code> , <code>StringEscapeUtils</code> , <code>PassThroughCorrelationConfigDataHolder</code>	— removed —	First-cut scope. Can be reintroduced inside <code>process()</code> without affecting the engine contract
State	<code>WorkerState</code> enum with <code>CREATED</code> , etc.	Simple <code>volatile boolean responseSent</code>	The worker runs inline on a VT; there are no queue stages to track

6.2 Field changes

Field	ServerWorker	VTBlockingServerWorker
Request	<code>SourceRequest request</code> (PT type)	<code>ClassicHttpRequest httpRequest</code> (HC5)
Response	— (built via <code>SourceResponseFactory</code> on demand)	<code>ClassicHttpResponse httpResponse</code> — pre-created by HttpCore 5
Context	Constructed internally	<code>HttpContext httpContext</code> — supplied by HttpCore 5, holds remote/local addresses set by the listener
Shared config	<code>SourceConfiguration</code>	IDENTICAL
Service maps	(lives on the <code>SourceConfiguration</code> / listener)	Passed in as constructor args for lookup convenience
Correlation / WSDL	<code>initiationTimestamp</code> , <code>correlationId</code> , <code>httpGetRequestProcessor</code> , <code>queuedTime</code> , <code>os</code>	— removed —

6.3 Function-level comparison

run() (NIO, on worker pool) → process() (VT, in-line)

`ServerWorker` is `Runnable` and is submitted to a pool. Its `run()` method walks the request, creates a `MessageContext`, and calls `AxisEngine.receive()`. It does *not* block on the response — the response is posted back asynchronously via `submitResponse()` called from `PassThroughHttpSender`.

`VTBlockingServerWorker.process()` is invoked synchronously by HttpCore 5's request handler; the whole mediation flow runs on this VT, including the backend call and response write. Structurally the method does:

- Parse request metadata** from `ClassicHttpRequest`: method, URI, case-insensitive header map, duplicate-header detection into `excessHeaders`.
- Build MessageContext** (`createMessageContext()`) — sets transport in / out, server-side flag, incoming transport name, transport headers, remote address from `HttpContext`.
- Mark REST vs SOAP vs entity-enclosing** and call one of `processEntityEnclosingRequest(true)` / `processNonEntityEnclosingRESTHandler(..., true)`. Both finish with `AxisEngine.receive(msgContext)`, which fires the entire Synapse mediation sequence on this VT.
- Safety net** in `finally`: if the sender never populated a response (e.g. exception), write a default one so HttpCore 5 has something to serialise. **500** + plain-text if the last-resort call itself fails.
- Thorough cleanup**:
 - Mark IN and OUT operation contexts `setComplete(true)` and `cleanup()`
 - Remove `ServiceGroupContext` from the `ConfigurationContext` cache
 - Strip heavyweight properties (`PASS_THROUGH_PIPE`, `TRANSPORT_HEADERS`, ...)
 - `MessageContext.destroyCurrentMessageContext()` — clears the `ThreadLocal`

Why cleanup is explicit here. In NIO, the reactor thread is long-lived and Axis2 releases state as connection events arrive. In VT, the VT ends when `process()` returns — but the `MessageContext ThreadLocal` and the `OperationContext` references on `ConfigurationContext` are not automatically reaped (they'd leak the whole Synapse config). Explicit cleanup is the only reliable hook.

NEW `submitResponse(MessageContext responseMsgCtx)`

Called by `VTHttpSender.submitResponse()` during the `respond` phase. Writes back onto the `ClassicHttpResponse`:

- Status code — from `PassThroughConstants.HTTP_SC` (Integer or String)
- Reason phrase — from `PassThroughConstants.HTTP_SC_DESC`

3. Transport headers — skipping `VTConstants.HOP_BY_HOP_HEADERS`
4. Body — three branches:
 - `NO_ENTITY_BODY=true` → just status + headers
 - `VT_INPUT_STREAM_PIPE` present **and** builder not invoked → `bodyStream.readAllBytes()` into `ByteArrayEntity` (pass-through response)
 - Otherwise — build via `MessageFormatterDecoratorFactory.createMessageFormatterDecorator()` and `formatter.writeTo()`
5. Set `responseSent = true` so the `finally` safety net doesn't double-write.

REUSED LOGIC `processEntityEnclosingRequest()`

The content-type inference, SOAP envelope bootstrap, REST vs SOAP routing logic is ported nearly verbatim from PassThrough's `ServerWorker`. Key change: after identifying the body, it wraps `httpRequest.getEntity().getContent()` in a `VTInputStreamPipe` and stashes it on `PASS_THROUGH_PIPE`.

REUSED LOGIC `handleRESTUrlPost()`

Port of `ServerWorker`'s identically-named method. Uses `RequestURIBasedDispatcher`, `PassThroughConfiguration`, `TransportUtils.createSOAPMessage()` — identical to PassThrough's multi-tenant-aware REST URL handling.

REUSED LOGIC `processHttpRequestUri()`

Calls `NhttpUtil.getRestUrlPostfix()` and constructs the `SERVICE_PREFIX` attribute — the exact pattern used by PassThrough. Difference: the local address/port come from `HttpContext` attributes (`CTX_LOCAL_ADDRESS`, `CTX_LOCAL_PORT`) set by the listener instead of being read off `NHttpServerConnection`.

`createMessageContext()`

Aspect	ServerWorker	VTBlockingServerWorker
Non-blocking flag	<code>CLIENT_API_NON_BLOCKING = TRUE</code>	<code>CLIENT_API_NON_BLOCKING = FALSE</code>
Transport OUT selection	TransportOutDescription from SourceConfiguration	Try VT transport-out first; fall back to <code>Constants.TRANSPORT_HTTP</code>
Transport IN	Source's TransportInDescription	Try HTTP first; fall back to source's; set <code>IncomingTransportName = http</code>
Headers	TreeMap, case-insensitive — same pattern	IDENTICAL
Remote address	From <code>HttpInetConnection.getRemoteAddress()</code>	From <code>HttpContext.getAttribute(CTX_REMOTE_ADDRESS)</code>

6.4 Removed ServerWorker features (and impact)

Feature	Impact
Correlation logs, correlation header, tenant-info initiators	Not exported to VT path yet. Easy to re-add by copying the relevant block into <code>process()</code> .
SSL cert-chain extraction	VT doesn't currently expose HTTPS; covered by future work (separate <code>VTPassThroughHttpSSLListener</code>).
WSDL / HTTP-GET processor (<code>httpGetRequestProcessor</code>)	WSDL-on-GET is not exercised in the initial deployments targeting VT. Fall-through goes to <code>processNonEntityEnclosingRESTHandler()</code> .
Wire logging via <code>SynapseDebugInfoHolder</code>	Replaceable with <code>HttpCore 5</code> logging interceptors.
State / timing instrumentation (<code>WorkerState</code> , <code>queuedTime</code>)	VT has no queue time — the worker runs immediately on accept. Obsolete.

7. `VTInputStreamPipe` VS `Pipe`

Files `vt/VTInputStreamPipe.java` — **50 LOC**
`passthru/Pipe.java` — **756 LOC**

The single biggest code-size reduction in the VT transport, and the clearest expression of "VT blocks, so we don't need async machinery". Summary of what `Pipe` does and why each piece disappears under VT:

Pipe concern	Why it exists in NIO	What replaces it under VT
<code>ControlledByteBuffer</code> + fixed-size ring	Producer (decoder callback) and consumer (formatter) run on different reactor threads; a bounded buffer is needed for back-pressure	Nothing — VT reads directly from the socket's <code>InputStream</code>
<code>IOControl</code> producerIoControl / consumerIoControl	Used to <code>requestInput()</code> / <code>suspendInput()</code> on the reactor	Nothing — VT has no reactor
<code>Lock lock = new ReentrantLock(); readCondition, writeCondition</code>	Producer and consumer synchronise here	Nothing — single VT does the whole flow
<code>DEFAULT_TIME_OUT_VALUE = 180000</code>	Prevents deadlock if a producer stalls	Socket-level <code>soTimeout</code> on the <code>ServerSocket</code>
<code>TruncatedChunkException</code> handling	Manual chunked decoding	<code>HttpCore 5</code> owns chunked decoding
<code>ContentDecoder</code> , <code>ContentEncoder</code> APIs	Core of the NIO I/O model	Nothing — blocking streams

`VTInputStreamPipe` is the entire replacement:

```
public class VTInputStreamPipe {
    private final InputStream inputStream;
    public VTInputStreamPipe(InputStream inputStream) { this.inputStream = inputStream; }
    public InputStream getInputStream() { return inputStream; }
}
```

Why it is still wrapped instead of storing the raw `InputStream`: the Synapse mediator code checks `msgContext.getProperty(PASS_THROUGH_PIPE)` and branches on type. Having a dedicated wrapper class makes `instanceof VTInputStreamPipe` checks in `VTHttpSender` and `VTBlockingServerWorker` unambiguous — we never confuse it with a bare `InputStream` accidentally set elsewhere.

8. `VirtualThreadWorkerPool` (new adapter)

File `vt/VirtualThreadWorkerPool.java`
Implements `org.apache.axis2.transport.base.threads.WorkerPool`

Axis2's `WorkerPool` interface is what the engine uses to submit `Runnable`s (mediator dispatches, error handlers, timers, etc.). The default impl, `NativeWorkerPool`, wraps a `ThreadPoolExecutor` with a fixed worker count. That bottlenecks the entire engine once the pool is saturated.

`VirtualThreadWorkerPool` satisfies the same interface but backs it with a virtual-thread-per-task executor (typically the one owned by the VT listener). Every `execute(Runnable)` spawns a fresh VT:

```
public class VirtualThreadWorkerPool implements WorkerPool {
    private final ExecutorService executor;
    public VirtualThreadWorkerPool(ExecutorService executor) { this.executor = executor; }
    public void execute(Runnable task) { executor.submit(task); }
    public int getActiveCount() { return -1; } // VTs don't expose this
    public int getQueueSize() { return 0; } // no queue - immediate dispatch
    public void shutdown(int timeout) {
        executor.shutdown();
        try { executor.awaitTermination(timeout, TimeUnit.SECONDS); }
        catch (InterruptedException e) { Thread.currentThread().interrupt(); }
    }
}
```

Why no direct counterpart: NIO uses `NativeThreadFactory` directly against a `ThreadPoolExecutor`. That mapping doesn't make sense here — VTs are so cheap that queueing is pointless.

Why `getActiveCount()` returns `-1`: the JDK doesn't expose live counts for VT executors without expensive thread-dump APIs. The metric is used only for diagnostics, so `-1` is the documented "unknown" sentinel.

9. `VTConstants` VS `PassThroughConstants`

`VTConstants` is small (120 LOC) and *additive* — it holds the few keys and defaults that are genuinely VT-only. Everything shared with the NIO transport (HTTP method names, `PASS_THROUGH_PIPE`, `MESSAGE_BUILDER_INVOKED`, `REST_URL_POSTFIX`, status keys, etc.) is reused directly from `PassThroughConstants`.

9.1 What `VTConstants` adds

Category	Constant	Purpose
MessageContext property keys	VT_SOURCE_REQUEST	Reserved for future parallelism with PT's source-request metadata
	VT_SOURCE_CONFIGURATION	Worker puts the <code>SourceConfiguration</code> here for sender access
	VT_SOURCE_CONNECTION	Reserved for per-connection debug tooling
	VT_TARGET_CONFIGURATION	Parallel to <code>VT_SOURCE_CONFIGURATION</code> on the outbound side
	VT_INPUT_STREAM_PIPE	Alternate to <code>PASS_THROUGH_PIPE</code> when the pipe must be named explicitly (e.g. response-body pass-through)
Transport wiring	VT_BLOCKING_CALL	Flag read by <code>VTHttpSender.invoke()</code> when invoked by <code>VTBlockingMsgSender</code>
	VT_TRANSPORT_SENDER	ConfigurationContext key the sender registers itself under so <code>CallMediator.init()</code> can look it up
Listener params	PARAM_PORT, PARAM_BIND_ADDRESS, PARAM_HOSTNAME, PARAM_BACKLOG, PARAM_SO_TIMEOUT, PARAM_KEEP_ALIVE_TIMEOUT, PARAM_CONNECT_TIMEOUT, PARAM_TCP_NODELAY, PARAM_SO_RCVBUF, PARAM_SO_SNDBUF, PARAM_MAX_REQUEST_SIZE	Mirrors the NIO param names but names them in one place to prevent typos
Defaults	DEFAULT_BACKLOG=1024, DEFAULT_SO_TIMEOUT=60000, DEFAULT_KEEP_ALIVE_TIMEOUT=115000, DEFAULT_CONNECT_TIMEOUT=10000, DEFAULT_TCP_NODELAY=true, STREAM_BUFFER_SIZE=8192	Standard HTTP server defaults; 115s idle keep-alive matches Nginx/Jetty conventions
Hop-by-hop set	HOP_BY_HOP_HEADERS = Set.of(transfer-encoding, content-length, connection, keep-alive, proxy-authenticate, proxy-authorization, te, trailers, upgrade)	RFC 7230 §6.1. Used by <code>VTBlockingServerWorker.submitResponse()</code> to drop headers HttpCore 5 will regenerate
HttpContext keys	CTX_REMOTE_ADDRESS, CTX_LOCAL_ADDRESS, CTX_LOCAL_PORT	Attribute names the listener uses to bridge Java-socket metadata into HttpCore 5 context for the worker
VT name prefix	VT_THREAD_PREFIX = "vt-passthru-"	Makes VT names discoverable in thread dumps

9.2 What VTConstants reuses directly from PassThroughConstants

```

HTTP_GET // method discrimination
HTTP_DELETE
REST_GET_DELETE_INVOKE
REST_URL_POSTFIX
SERVICE_PREFIX
EPR_TO_SERVICE_NAME_MAP
SERVICE_URI_LOCATION
PASS_THROUGH_TRANSPORT_WORKER_POOL
PASS_THROUGH_PIPE
MESSAGE_BUILDER_INVOKED
HTTP_SC // response status
HTTP_SC_DESC
NO_ENTITY_BODY
APPLICATION_OCTET_STREAM
REST_REQUEST_CONTENT_TYPE
IGNORE_SC_ACCEPTED
CORRELATION_LOGGER
REQUEST_CONTENT_TYPE

```

Why not redefine these: they are part of the public property-key contract between transport and mediators. Re-declaring them would silently fork the contract — mediators reading `PASS_THROUGH_PIPE` would not see our pipe. The intent is explicit: VT piggy-backs on the same property-key vocabulary.

10. Reused Existing Utilities (Cross-Cutting)

The table below consolidates *all* pre-existing Synapse/Axis2 classes the VT transport leans on unchanged. Demonstrates that VT is a *surgical* insertion — not a rewrite.

Reused type	Consumer	Role
SourceConfiguration	Listener, Worker, Sender	Holds transport params, metrics, WorkerPool, scheme, EPR prefixes
TargetConfiguration	Sender	Same for outbound; powers <code>PassThroughTransportUtils.removeUnwantedHeaders</code>
PassThroughTransportMetricsCollector	Listener, Sender	JMX-style counters
AxisServiceTracker, AxisServiceFilter, AxisServiceTrackerListener	Listener	EPR map updates on service deploy/undeploy
SessionContextUtil	Listener (<code>getSessionContext</code>)	Session affinity for sticky transports
NhttpUtil (<code>getRestUrlPostfix</code> , ...)	Worker	URI → service-prefix / REST-postfix split
RESTUtil.dispatchAndVerify	Worker (<code>handleRESTUrlPost</code>)	Populate Axis2 operation/service from REST URI
HTTPTransportUtils.isRESTRequest, initializeMessageContext	Worker	Content-type classification + SOAP version selection
BuilderUtil.getCharsetEncoding	Worker	Charset extraction from <code>Content-Type</code>
PassThroughTransportUtils.removeUnwantedHeaders	Sender	Strip <code>Content-Length</code> , etc. before send
PassThroughTransportUtils.getOMOutputFormat	Worker (<code>submitResponse</code>)	OMOutputFormat with transport-aware charset settings
MessageFormatterDecoratorFactory	Worker	Correct <code>MessageFormatter</code> (JSON, XML, MIME) per content type
AbstractHTTPSender / HTTPSender (Axis2)	Sender (<code>writeMessageWithCommons</code>)	OM-tree fallback path when streaming is not applicable
ServletBasedOutTransportInfo, AxisHttpResponseImpl	Sender (<code>sendUsingOutputStream</code>)	Servlet / JAX-WS compatibility when the response is written to a provided OutputStream
MessageProcessorSelector	Sender	Formatter lookup when writing responses via OutputStream
TransportUtils (<code>doWriteMTOM</code> , <code>doWriteSwA</code> , <code>isDoingREST</code> , ...)	Sender	Standard format flags
AddressingHelper	Sender	<code>isReplyRedirected</code> check
JavaUtils (<code>isTrueExplicitly</code> , ...)	Sender	Safe boolean parsing
PassThroughConstants (full enum of keys)	All three files	See §9.2
Scheme	Listener, Sender	Axis2 scheme descriptor (name, default port, secure flag)
NhttpConstants.EXCESS_TRANSPORT_HEADERS, REMOTE_HOST	Worker	Duplicate-header slot and remote-host attribute

11. Deployment Notes & Risks

11.1 axis2.xml configuration

```
<transportReceiver name="vt-http"
    class="org.apache.synapse.transport.passthru.vt.VTPassThroughHttpListener">
  <parameter name="port">8300</parameter>
  <parameter name="so_timeout">60000</parameter>
  <parameter name="backlog">1024</parameter>
</transportReceiver>

<transportSender name="http"
    class="org.apache.synapse.transport.passthru.vt.VTHttpSender">
  <parameter name="PROTOCOL">HTTP/1.1</parameter>
  <parameter name="Transfer-Encoding">chunked</parameter>
  <parameter name="cacheHttpClient">true</parameter>
  <parameter name="defaultMaxConnPerHost">500</parameter>
  <parameter name="maxTotalConnections">1000</parameter>
</transportSender>
```

11.2 axis2_blocking_client.xml (for CallMediator blocking path)

```
<transportSender name="http"
    class="org.apache.synapse.transport.passthru.vt.VTHttpSender">
  <parameter name="PROTOCOL">HTTP/1.1</parameter>
  <parameter name="Transfer-Encoding">chunked</parameter>
  <parameter name="cacheHttpClient">true</parameter>
</transportSender>
```

Mandatory: `cacheHttpClient=true`. Without it `VTHttpSender.init()` does not build the pooled `CloseableHttpClient`, and `sendStreamedRequest()` throws an `AxisFault`.

11.3 Known limitations (first cut)

- **HTTPS not yet wired** — SSL listener is a follow-up.
- **JMX MBeans not registered** — metrics object is created but not exposed.
- **Stream interceptors / wire logging** not wired — can be added as HttpCore 5 interceptors.
- **WSDL GET-request processor** not integrated — affects `?wsdl` endpoints on the VT listener only.
- **Inbound endpoint managers** calling `PassThroughInboundEndpointHandler.getPassThroughSourceConfiguration()` still expect the NIO transport. Running VT alongside NIO avoids this; VT-only deployments need the NIO listener disabled **and** the inbound endpoint manager updated to ask for the VT source configuration.
- **Correlation log / tenant initiator** is a trivial backport once the core flow is green.

11.4 Behavioural parity with NIO (validated by branch-level tests)

- Zero-copy body streaming triggers on the same mediator shapes as NIO (`<call/>` without content-aware predecessor). See `CallMediator.isContentAware()=false`.
- A `<log/>` (any level except `message_template` / `custom`) before `<call/>` forces an OM build, same as NIO.
- Response body from the backend flows through `TRANSPORT_IN` identically to `AbstractHTTPSender`.